

## **Sistema de Inventario y Ventas para Tienda**

### **Objetivo General**

Desarrollar un sistema web en Java que permita gestionar el inventario y las ventas de una tienda, facilitando el registro, actualización y control de productos, usuarios y ventas mediante operaciones CRUD y una base de datos relacional.

### **Objetivos Específicos**

1. Implementar un módulo de autenticación para el acceso seguro de los usuarios al sistema.
2. Desarrollar funcionalidades CRUD para la gestión de productos, categorías y proveedores.
3. Permitir el registro y control de ventas realizadas dentro de la tienda.
4. Gestionar automáticamente el stock de productos después de cada venta.
5. Diseñar una base de datos relacional que almacene de manera organizada la información del sistema.
6. Crear una interfaz web amigable e intuitiva para facilitar el uso del sistema.
7. Utilizar herramientas de control de versiones para almacenar y administrar el código fuente del proyecto.

## **Justificación**

Actualmente, muchos pequeños negocios realizan el control de inventario y ventas de manera manual, lo que puede generar pérdidas de información, errores en los registros y dificultades para llevar un control adecuado del stock disponible.

El desarrollo de un sistema web de inventario y ventas permitirá optimizar la administración de productos y ventas de la tienda, mejorando la organización de la información y reduciendo errores humanos. Además, el sistema facilitará el acceso rápido a los datos y permitirá automatizar procesos importantes como la actualización del inventario después de cada venta.

Este proyecto también permitirá aplicar los conocimientos adquiridos en programación Java Web, bases de datos relacionales, modelado de sistemas y desarrollo de aplicaciones CRUD, fortaleciendo las habilidades prácticas en el desarrollo de software.

## **Entorno de Trabajo**

### **Lenguaje de Programación**

- Java

### **Framework / Tecnologías Web**

- Java Web
- JSP
- Servlets
- HTML5
- CSS3
- Bootstrap

### **Base de Datos**

- MariaDB o MySQL

### **Herramientas de Desarrollo**

- Apache NetBeans / IntelliJ IDEA / Eclipse
- HeidiSQL

### **Arquitectura**

- Modelo Cliente-Servidor
- Arquitectura MVC (Modelo Vista Controlador)

## **Requerimientos Funcionales**

### **Gestión de Usuarios**

1. El sistema debe permitir el inicio de sesión mediante usuario y contraseña.
2. El sistema debe permitir cerrar sesión de forma segura.
3. El administrador podrá registrar nuevos usuarios.
4. El administrador podrá editar la información de los usuarios.
5. El administrador podrá eliminar usuarios del sistema.

### **Gestión de Productos**

6. El sistema debe permitir registrar nuevos productos.
7. El sistema debe permitir consultar la lista de productos disponibles.
8. El sistema debe permitir editar la información de los productos.
9. El sistema debe permitir eliminar productos del inventario.
10. El sistema debe permitir buscar productos por nombre o categoría.
11. El sistema debe mostrar la cantidad disponible de cada producto.

### **Gestión de Categorías**

12. El sistema debe permitir registrar categorías de productos.
13. El sistema debe permitir editar categorías.
14. El sistema debe permitir eliminar categorías.

### **Gestión de Proveedores**

15. El sistema debe permitir registrar proveedores.
16. El sistema debe permitir consultar la información de los proveedores.
17. El sistema debe permitir editar los datos de los proveedores.
18. El sistema debe permitir eliminar proveedores.

### **Gestión de Ventas**

19. El sistema debe permitir registrar ventas de productos.
20. El sistema debe calcular automáticamente el total de la venta.
21. El sistema debe actualizar automáticamente el stock después de cada venta.
22. El sistema debe generar un historial de ventas realizadas.
23. El sistema debe permitir consultar ventas por fecha.

## **Reportes**

- 24. El sistema debe permitir visualizar reportes básicos de productos y ventas.
- 25. El sistema debe mostrar productos con bajo stock.

## **Requerimientos No Funcionales**

### **Seguridad**

1. Las contraseñas de los usuarios deben almacenarse de manera segura.
2. Solo los usuarios autorizados podrán acceder al sistema mediante autenticación.
3. El sistema debe restringir funcionalidades según el rol del usuario.

### **Usabilidad**

6. La interfaz del sistema debe ser intuitiva y fácil de usar.
7. El sistema debe contar con formularios organizados y navegación clara.

### **Compatibilidad**

8. El sistema debe funcionar en navegadores modernos como Google Chrome, Microsoft Edge y Firefox.
9. El sistema debe ser compatible con sistemas operativos Windows y Linux.

### **Mantenimiento**

10. El código fuente debe estar organizado bajo el patrón MVC para facilitar el mantenimiento.
11. El proyecto debe almacenarse en un repositorio GitHub para el control de versiones.

### **Disponibilidad**

12. El sistema debe estar disponible mientras el servidor Apache Tomcat esté en ejecución.

### **Base de Datos**

13. La base de datos debe garantizar integridad y consistencia de la información almacenada.
14. El sistema debe evitar duplicidad de datos en registros importantes.

DCU's